**Paradigmas y Lenguajes de Programación**

**Carrera:** Licenciatura en Gestión de Recursos Tecnológicos
Universidad Gastón Dachary

Año 2026

**Nombre y apellido**: Cristian Collins
**Número de matrícula/usuario:** ugd600249
**Carrera:** Licenciatura en Recursos Tecnológicos
**Materia**: Paradigmas y Lenguajes de Programación
**Profesores**: Ing. Agustín Encina, Ing. Dante Sicardi

UNIVERSIDAD Gastón Dachary

**ACTIVIDAD N° 2**

# Lenguaje elegido: Prolog

Para la presente actividad se seleccionó el lenguaje de programación Prolog, perteneciente al paradigma lógico, dentro de la familia de los lenguajes declarativos.

Prolog se basa en la definición de hechos y reglas que describen un dominio de problema. A partir de esa base de conocimiento, el motor de inferencia del lenguaje deduce automáticamente las soluciones posibles mediante un proceso de búsqueda con backtracking.

# Paradigma: Lógico (Declarativo)

El paradigma lógico se caracteriza por describir *qué condiciones debe cumplir una solución*, en lugar de indicar paso a paso *cómo obtenerla*.

A diferencia del paradigma imperativo:

- No se trabaja con estados mutables ni variables de control
- No se definen algoritmos secuenciales
- La resolución del problema se delega al motor de inferencia

Esto permite construir soluciones más expresivas, cercanas al dominio del problema y fácilmente extensibles ante nuevos requerimientos.

# Ejemplo desarrollado: Generación automática de horarios laborales

Se implementó un sistema en Prolog que genera automáticamente un cronograma semanal de trabajo para un conjunto de empleados de una empresa con múltiples sectores.

El sistema contempla:

- Días de la semana y turnos (mañana, tarde, noche)
- Sectores de trabajo y oficios requeridos por cada uno
- Habilidades individuales de cada empleado
- Días de franco semanal
- Restricciones de cobertura mínima por turno y por sector

# Fragmento representativo del código

A continuación se presenta un fragmento del código desarrollado, fiel a la implementación real:

```
% Empleados y sus habilidades
empleado(e1).  empleado(e2).  empleado(e3).
sabe(e1, cocinero).     sabe(e1, vendedor).
sabe(e2, panadero).     sabe(e2, vendedor).
sabe(e3, fabrica_pasta). sabe(e3, vendedor).

% Franco semanal
franco(e1, lunes).

% Relacion sector-oficio
sector_requiere(venta, vendedor).

% Condicion para cubrir un sector
puede_cubrir(Empleado, Sector, Turno, Dia) :-
    disponible(Empleado, Dia),
    sector_requiere(Sector, Oficio),
    sabe(Empleado, Oficio),
    sector_habilitado(Sector, Turno).

% Generacion del plan semanal completo
generar_plan(Plan) :-
    dias_semana(Dias),
    generar_plan_dias(Dias, Plan),
    plan_ok(Plan).

generar_plan_dias([], []).
generar_plan_dias([Dia|RestoDias], PlanCompleto) :-
    generar_asignaciones_dia(Dia, AsignacionesDia),
    generar_plan_dias(RestoDias, PlanResto),
    append(AsignacionesDia, PlanResto, PlanCompleto).
```

El predicado **generar_plan/1** itera sobre todos los días de la semana, genera las asignaciones de cada uno y las acumula en un plan completo que luego es validado por **plan_ok/1**.

# Reglas del sistema

El sistema garantiza que:

- Cada empleado trabaja únicamente en sectores compatibles con su oficio
- Ningún empleado trabaja en su día de franco
- Ningún empleado tiene más de un turno por día
- Existe cobertura en los turnos de mañana, tarde y noche todos los días
- Todos los días hay al menos un empleado asignado al sector de venta en turno mañana o tarde
- No se producen asignaciones duplicadas o superpuestas para el mismo empleado en el mismo día

# Consultas realizadas

Se realizaron las siguientes consultas sobre el sistema para generar y analizar el cronograma:

```
% Genera y muestra el plan completo
?- ver_plan.

% Obtiene el plan como estructura de datos
?- generar_plan(Plan).

% Horario de un empleado especifico
?- generar_plan(Plan), horario_empleado(Plan, e1, Horario).

% Empleados que trabajan un dia y turno determinados
?- generar_plan(Plan), quienes_trabajan(Plan, domingo, tarde, Lista).
```

# Resultados obtenidos

El sistema genera automáticamente un cronograma semanal válido, asignando a cada empleado disponible un turno y un sector acorde a sus habilidades, respetando todas las restricciones definidas.

*Ejemplo de salida:*

```
lunes   | manana | e2 | panaderia
lunes   | tarde  | e4 | comidas_elaboradas
```

lunes | noche | e5 | pastas

...

domingo | tarde | e3 | venta

# Análisis

El ejemplo desarrollado ilustra con claridad las ventajas del paradigma lógico aplicado a un problema de planificación y asignación de recursos.

En lugar de programar un algoritmo paso a paso, se modela el dominio del problema mediante hechos y reglas: empleados, turnos, sectores, habilidades y restricciones. A partir de esa base, Prolog genera automáticamente soluciones válidas mediante su motor de inferencia y el mecanismo de **backtracking**, explorando distintas combinaciones posibles hasta encontrar aquellas que satisfacen todas las condiciones.

Las validaciones como sin_superposicion/1 y plan_ok/1 actúan como filtros declarativos: no describen cómo evitar los conflictos, sino que simplemente establecen que no deben existir. El motor de Prolog se encarga de descartar cualquier asignación que los viole. El uso de reglas como plan_ok/1 permite centralizar la validación del sistema, separando la generación de soluciones de su verificación, lo cual es característico del paradigma declarativo.

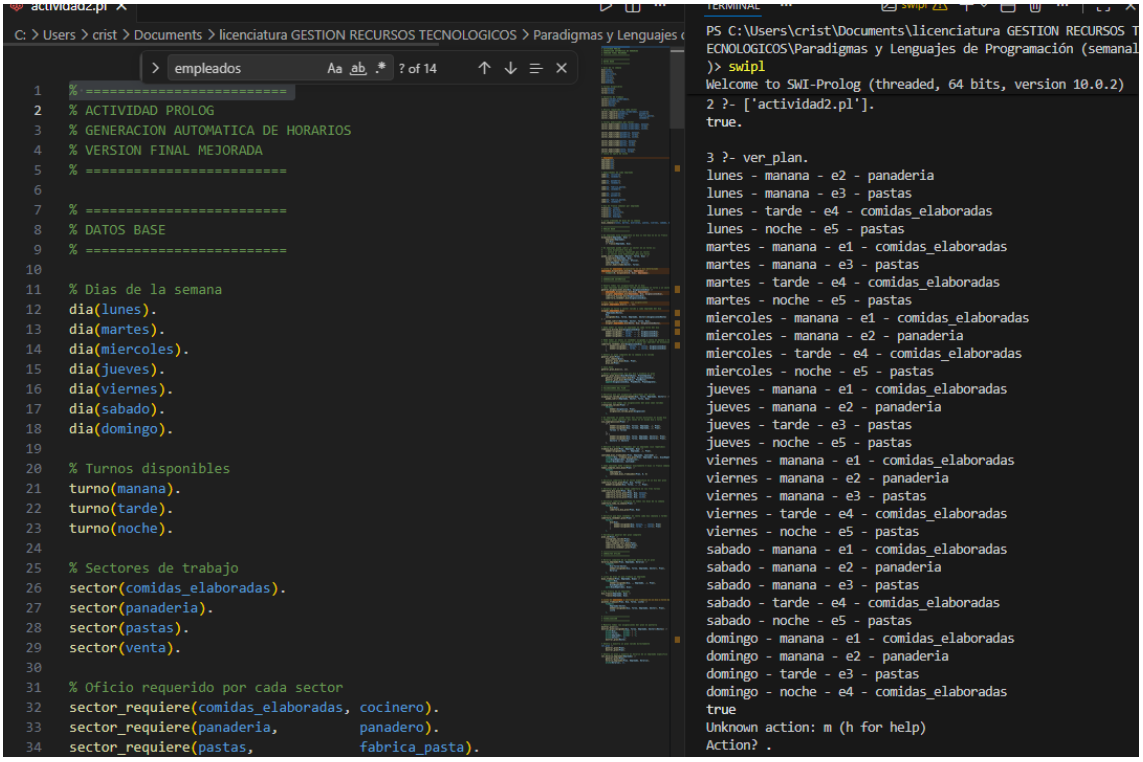
En lenguajes imperativos como Java o Python, este problema requeriría implementar manualmente algoritmos de asignación, estructuras de datos para el control de restricciones y lógica de backtracking propia. En Prolog, en cambio:

- Se define el conocimiento del problema
- Se especifican las condiciones de validez
- El sistema encuentra las soluciones automáticamente

Esto produce una solución más concisa, legible y fácilmente adaptable ante cambios en las reglas del negocio.

# Conclusión

El paradigma lógico permite abordar problemas complejos mediante una representación declarativa del conocimiento, enfocándose en las condiciones que debe cumplir una solución en lugar de los pasos necesarios para obtenerla.

El sistema de generación automática de horarios desarrollado en Prolog demuestra cómo, a partir de hechos y reglas bien definidos, es posible construir un cronograma válido de manera completamente automática, sin necesidad de programar explícitamente el proceso de búsqueda.

Este enfoque resulta especialmente valioso en problemas de planificación, asignación de recursos, inteligencia artificial y sistemas expertos, donde la complejidad radica en las restricciones del problema más que en el procedimiento de resolución.